408 数据结构：图（Graph）完整复习笔记

这份笔记按照考研 408 的考查特点整理，重点放在：

1. **基本概念**：选择题常考，尤其是度、边数、连通性。
2. **图的存储结构**：邻接矩阵、邻接表的空间复杂度和操作复杂度。
3. **图的遍历**：DFS、BFS，尤其是遍历序列和时间复杂度。
4. **最小生成树**：Prim、Kruskal 算法。
5. **最短路径**：Dijkstra、Floyd 算法。
6. **拓扑排序**：拓扑序列、唯一性判断。
7. **关键路径**：AOE 网、四个时间参数的计算。
8. **算法设计题**：常见代码模板和答题套路。

其中，拓扑排序、最短路径、图的遍历和算法设计值得重点掌握。

一、图的基本概念

1. 图的定义

图由顶点集合和边集合组成：

$$G = (V, E)$$

- V：顶点集合。
- E：边集合。
- $n = |V|$：顶点数。
- $e = |E|$：边数。

按照边是否有方向，可以分为：

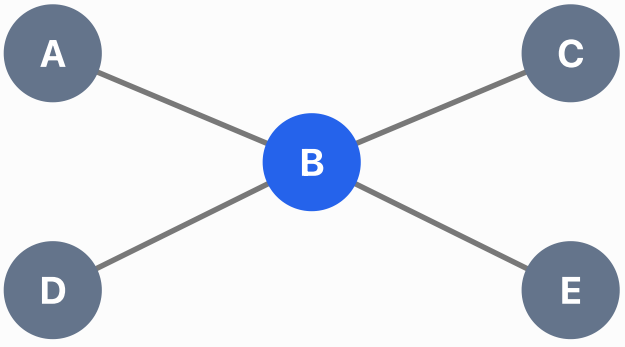
类型	特点
无向图	边没有方向，记为 (u, v)
有向图	边具有方向，记为 $\langle u, v \rangle$
带权图	边具有权值
无权图	边没有权值，计算路径长度时通常每条边按 1 计

2. 顶点的度

无向图

顶点的度 $TD(v)$：

$$TD(v) = \text{与顶点 } v \text{ 关联的边数}$$



顶点 B 与 4 条边关联，因此 TD(B)=4；其余顶点的度均为 1。

重要公式：

$$\boxed{\sum_{i=1}^{n} TD(v_i) = 2e}$$

原因：每条无向边会被计算两次。

因此：

$$\boxed{e = \frac{1}{2}\sum_{i=1}^{n} TD(v_i)}$$

考试结论：无向图中，度为奇数的顶点个数一定是偶数。

有向图

有向图中的度分为：

- 入度 $ID(v)$：指向顶点 v 的边数。
- 出度 $OD(v)$：从顶点 v 出发的边数。

$$TD(v) = ID(v) + OD(v)$$

重要公式：

$$\boxed{\sum ID(v) = \sum OD(v) = e}$$

$$\boxed{\sum TD(v) = 2e}$$

记忆：

- 无向图：度数之和 = 2 倍边数。
- 有向图：入度之和 = 出度之和 = 边数。

3. 完全图的边数

这是 408 高频计算考点。

无向完全图

任意两个不同顶点之间都有一条边。

$$\boxed{e = \frac{n(n-1)}{2}}$$

例如，5 个顶点：

$$e = \frac{5 \times 4}{2} = 10$$

有向完全图

任意两个不同顶点之间都有两条方向相反的弧。

$$\boxed{e = n(n-1)}$$

例如，5 个顶点：

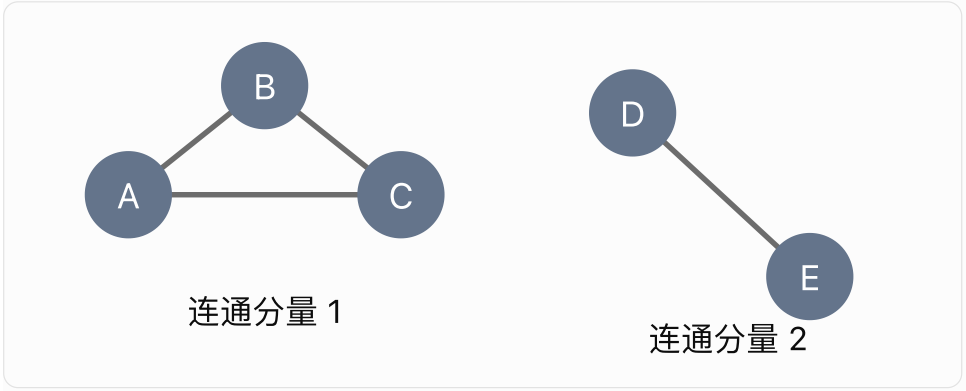
$$e = 5 \times 4 = 20$$

图的类型	最大边数（简单图）
无向图	$\frac{n(n-1)}{2}$
有向图	$n(n-1)$

4. 连通图与连通分量

无向图

- 连通：两个顶点之间存在路径。
- 连通图：任意两个顶点之间都有路径。
- 连通分量：无向图中的极大连通子图。



重要结论：

具有 n 个顶点的无向连通图，至少有 $n-1$ 条边。

$$\boxed{e_{\min} = n - 1}$$

如果无向图有 n 个顶点、$n-1$ 条边且连通，那么它一定是一棵树。

注意：只有 $n-1$ 条边不代表一定连通。

有向图

概念	含义
强连通	顶点 u 能到达 v，且 v 能到达 u
强连通图	任意两个顶点之间都能相互到达
强连通分量	极大强连通子图

对于 $n \geq 2$ 的简单有向图：

- 强连通图至少需要 n 条弧。
- n 个顶点组成一个有向环，即可实现强连通。

5. 路径与回路

名称	含义
路径	顶点之间依次经过相邻边形成的序列
路径长度	无权图中为边数；带权图中通常为权值之和
简单路径	顶点不重复的路径
回路（环）	起点和终点相同的路径
简单回路	除起点与终点相同外，其他顶点不重复

特别注意：

- 路径可以重复经过顶点，简单路径不可以。
- 经过不同边不一定是简单路径。
- 有向无环图（DAG）可以进行拓扑排序。

二、图的存储结构

408 主要考邻接矩阵和邻接表。

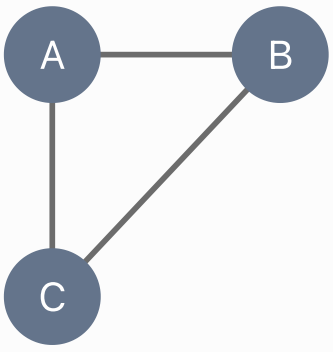
1. 邻接矩阵

使用二维数组存储图：

$$A[i][j] = \begin{cases} 1, & v_i \text{ 与 } v_j \text{ 之间有边} \\ 0, & \text{否则} \end{cases}$$

对于带权图，通常用边权代替 1，用 ∞ 表示不同顶点之间不存在边。

示例



	A	B	C
A	0	1	1
B	1	0	0
C	1	0	0

邻接矩阵的性质

无向图：

邻接矩阵一定对称：

$$A[i][j] = A[j][i]$$

第 i 行元素之和为顶点 i 的度。

有向图：

- 第 i 行元素之和 = 出度。
- 第 i 列元素之和 = 入度。

记忆：

> 行出列入。

邻接矩阵的复杂度

操作	时间复杂度
存储空间	$O(n^2)$
判断两顶点是否有边	$O(1)$
求某顶点的全部邻接点	$O(n)$
遍历整张图	$O(n^2)$

适合：**稠密图**。

邻接矩阵的重要性质：矩阵幂

设 A 是无权图的邻接矩阵。

$$\boxed{A^k[i][j]}$$

表示从顶点 i 到顶点 j，**长度恰好为 k 的步行（Walk）数量**。

注意：

- 可以重复经过顶点和边。
- 不要求是简单路径。

例如：

$$A^2[i][j]$$

表示从 i 到 j 恰好经过两条边的步行数量。

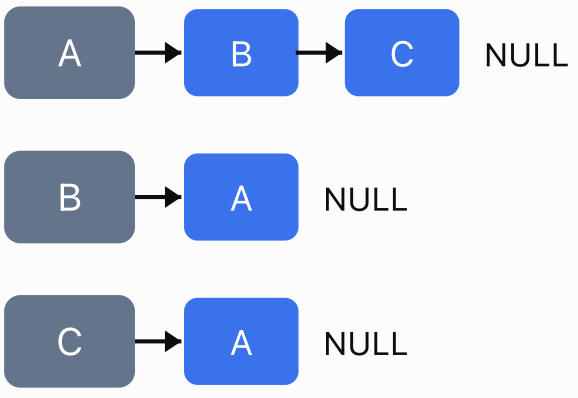
这也是 408 可能考查的结论。

2. 邻接表

邻接表由两部分组成：

1. 顶点表：存储所有顶点。
2. 边表：每个顶点对应一个链表，记录与其相邻的顶点。

例如：



邻接表记录每个顶点所连接的邻接点。邻接点在链表中的排列顺序不一定唯一。

邻接表的性质

无向图：

一条边会存储两次。

因此边表结点总数为：

$$\boxed{2e}$$

有向图：

一条弧通常只存储一次。

因此边表结点总数为：

$$\boxed{e}$$

若采用出边邻接表：

- 求出度容易。
- 求入度需要扫描所有边表。

邻接表的复杂度

操作	时间复杂度
存储空间	$O(n+e)$
查找某顶点的全部邻接点	$O(d(v))$
判断两顶点是否有边	最坏 $O(n)$
遍历整张图	$O(n+e)$

适合：**稀疏图**。

3. 邻接矩阵与邻接表比较

比较项目	邻接矩阵	邻接表
存储空间	$O(n^2)$	$O(n+e)$
稠密图	适合	不太适合
稀疏图	浪费空间	适合
判断两点是否相邻	$O(1)$	需查找
遍历全部邻接点	$O(n)$	$O(d(v))$
DFS/BFS 总复杂度	$O(n^2)$	$O(n+e)$

必背：

$$\boxed{\text{邻接矩阵：}O(n^2)}$$

$$\boxed{\text{邻接表：}O(n+e)}$$

4. 十字链表与邻接多重表

这两种结构主要考概念。

结构	适用图	特点
十字链表	有向图	同时方便查找入边和出边
邻接多重表	无向图	每条边只用一个边结点表示，便于边的操作

三、图的遍历

图的遍历主要有两种：

- DFS：深度优先搜索。

- BFS：广度优先搜索。

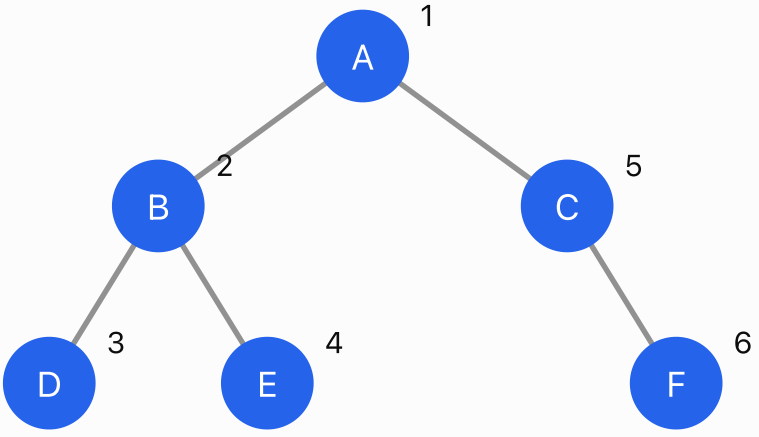
1. DFS 深度优先搜索

DFS 的核心思想：

一条路走到底，走不通再回退。

类似于树的先序遍历。

DFS 示例



假设按从左到右的邻接点顺序搜索，访问顺序为 A → B → D → E → C → F。

DFS 代码模板

```c
C

bool visited[MAXV];

void DFS(MGraph G, int v)
{
    visited[v] = true;
    visit(v);

    for (int w = 0; w < G.numVertices; w++)
    {
        if (G.Edge[v][w] != 0 && !visited[w])
        {
            DFS(G, w);
        }
    }
}
```

这里假设邻接矩阵用 0 表示无边，且图没有需要特殊处理的零权边。

DFS 复杂度

存储结构	时间复杂度
邻接矩阵	$O(n^2)$
邻接表	$O(n+e)$

递归辅助空间最坏：

$$\boxed{O(n)}$$

DFS 的常见应用

- 判断图是否连通。
- 判断是否存在路径。
- 检测环。
- 求连通分量。
- 拓扑排序。
- 寻找生成树。

2. BFS 广度优先搜索

BFS 的核心思想：

先访问距离近的顶点，再访问距离远的顶点。

类似于树的层序遍历。

使用队列实现。

BFS 示例

对于前面的图：

- 第 0 层：A
- 第 1 层：B、C
- 第 2 层：D、E、F

因此 BFS 序列为：

$$\boxed{A, B, C, D, E, F}$$

注意：具体序列与邻接点访问顺序有关。

BFS 代码模板

```c
C

void BFS(MGraph G, int v)
{
    bool visited[MAXV] = {false};
    int queue[MAXV];
    int front = 0, rear = 0;

    visited[v] = true;
    visit(v);
    queue[rear++] = v;

    while (front < rear)
    {
        int u = queue[front++];

        for (int w = 0; w < G.numVertices; w++)
        {
            if (G.Edge[u][w] != 0 && !visited[w])
            {
                visited[w] = true;
                visit(w);
                queue[rear++] = w;
            }
```

BFS 复杂度

存储结构	时间复杂度
邻接矩阵	$O(n^2)$
邻接表	$O(n+e)$

辅助空间：

$$\boxed{O(n)}$$

BFS 的重要性质

BFS 可以求无权图中从起点到其他各顶点的最短路径。

这里的最短路径是指：

经过的边数最少。

对于所有边权均为 1 的图，也可以使用 BFS 求最短路径。

3. DFS 与 BFS 对比

比较	DFS	BFS
核心思想	一条路走到底	一层一层访问
主要辅助结构	栈 / 递归	队列
对应树遍历	先序遍历	层序遍历
无权图最短路径	不保证	可以
邻接矩阵时间复杂度	$O(n^2)$	$O(n^2)$
邻接表时间复杂度	$O(n+e)$	$O(n+e)$

高频陷阱

1. DFS 和 BFS 的访问序列不一定唯一。
2. 访问序列取决于邻接点的选择顺序。
3. BFS 在无权图中可以求最短路径，DFS 不保证。
4. 非连通图需要从每个未访问的顶点重新开始遍历。
5. 无向图的 DFS 生成树和 BFS 生成树不一定相同。

四、最小生成树 MST

1. 基本概念

生成树：

包含无向连通图全部顶点、且无环的连通子图。

如果有 n 个顶点，生成树一定有：

$$\boxed{n-1 \text{ 条边}}$$

最小生成树（Minimum Spanning Tree，MST）：

所有生成树中，边权之和最小的生成树。

注意：

- MST 针对带权无向连通图。
- MST 不一定唯一。
- MST 的总权值一定相同。
- 若所有边权互不相同，则 MST 唯一。

2. Prim 算法

Prim 算法：

从一个顶点出发，不断选取连接已选顶点集合和未选顶点集合的最小权值边。

步骤：

1. 随便选择一个起点。

2. 找到连接当前生成树与树外顶点的最小边。
3. 把新顶点加入生成树。
4. 重复，直到全部顶点加入。

示例

设边权如下：

边	权值
A-B	2
A-C	5
B-C	1
B-D	4
C-D	3

从 A 出发：

步骤	选择边	已加入顶点
1	A-B（2）	A、B
2	B-C（1）	A、B、C
3	C-D（3）	A、B、C、D

最小生成树权值：

$$2+1+3=\boxed{6}$$

Prim 的关键：

只考虑一端在树内、另一端在树外的边。

Prim 复杂度

常见邻接矩阵实现：

$$\boxed{O(n^2)}$$

适合稠密图。

3. Kruskal 算法

Kruskal 算法：

将所有边按权值从小到大排序，依次选择不会形成环的边。

步骤：

1. 所有边按权值升序排列。
2. 选择当前权值最小的边。
3. 如果加入后形成环，则舍弃。
4. 直到选出 $n-1$ 条边。

使用并查集判断两个顶点是否已经连通。

示例

仍使用前面的图。

边权排序：

$$BC(1), AB(2), CD(3), BD(4), AC(5)$$

依次选择：

- BC：选择。
- AB：选择。
- CD：选择。

已经选出 3 条边，结束。

$$MST = 1 + 2 + 3 = 6$$

Kruskal 复杂度

采用排序和并查集：

$$\boxed{O(e \log e)}$$

适合稀疏图。

4. Prim 与 Kruskal 对比

比较项目	Prim	Kruskal
核心	不断扩充顶点集合	不断选择最小边
选边范围	树内到树外的边	全图尚未处理的边
是否需要判断成环	通常不需要额外判断	需要
常见辅助结构	lowcost 数组	并查集
常见复杂度	$O(n^2)$	$O(e \log e)$
适合	稠密图	稀疏图

考试口诀：

> Prim 看顶点，Kruskal 看边。

五、最短路径

408 主要掌握：

1. BFS：无权图最短路径。
2. Dijkstra：单源最短路径。
3. Floyd：所有顶点对之间的最短路径。

1. Dijkstra 算法

适用范围

求从某个源点到其他全部顶点的最短路径。

要求：

边权不能为负数。

核心思想

维护两个集合：

- 已确定最短路径的顶点集合。
- 尚未确定最短路径的顶点集合。

每次选择距离源点最近的未确定顶点，再更新其他顶点的距离。

关键公式：松弛操作

假设当前经过顶点 u 到达 v。

$$\boxed{dist[v] = \min(dist[v], dist[u] + w(u, v))}$$

意思是：

> 比较原来的距离与经过 u 绕过去的距离，取较小值。

这是最短路径算法最重要的公式之一。

示例

设图中存在以下有向边：

边	权值
A → B	2
A → C	6
B → C	1
B → D	5
C → D	2

求 A 到其他顶点的最短距离。

初始化：

$$dist = [0, 2, 6, \infty]$$

选择 B：

经过 B 更新 C：

$$dist[C] = \min(6, 2 + 1) = 3$$

更新 D：

$$dist[D] = \min(\infty, 2 + 5) = 7$$

选择 C：

$$dist[D] = \min(7, 3 + 2) = 5$$

最终：

顶点	最短距离	最短路径
A	0	A
B	2	A→B
C	3	A→B→C
D	5	A→B→C→D

Dijkstra 考试做题模板

1. 初始化距离数组。
2. 选择未确定顶点中距离最小的顶点。
3. 将该顶点标记为已确定。
4. 使用该顶点更新其他顶点的距离。
5. 重复，直到全部可达顶点处理完。

常见邻接矩阵实现：

$$\boxed{O(n^2)}$$

注意：

Dijkstra 不适合包含负权边的图，即使某些负权图上碰巧可以得到正确结果，也不具备普遍正确性。

2. Floyd 算法

Floyd 算法用于：

求任意两个顶点之间的最短路径。

核心公式

$$\boxed{D[i][j] = \min(D[i][j], D[i][k] + D[k][j])}$$

解释：

从 i 到 j：

- 要么直接走原有路线。
- 要么经过顶点 k。

取两者最小值。

三层循环模板

```c
for (int k = 0; k < n; k++)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (dist[i][k] != INF &&
                dist[k][j] != INF &&
                dist[i][k] + dist[k][j] < dist[i][j])
            {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}
```

必须记住：k 在最外层。

复杂度

时间：

$$\boxed{O(n^3)}$$

空间：

$$\boxed{O(n^2)}$$

Floyd 可以处理负权边，但不能正确处理涉及可达负权回路的最短路径问题。

3. 最短路径算法比较

算法	解决问题	负权边	典型复杂度
BFS	无权图单源最短路径	不适用	$O(n+e)$
Dijkstra	单源最短路径	不允许	$O(n^2)$
Floyd	所有顶点对最短路径	允许，但不能有影响结果的负权回路	$O(n^3)$

六、拓扑排序

这是 408 必须掌握的重点。

1. 基本概念

拓扑排序用于：

有向无环图（DAG）。

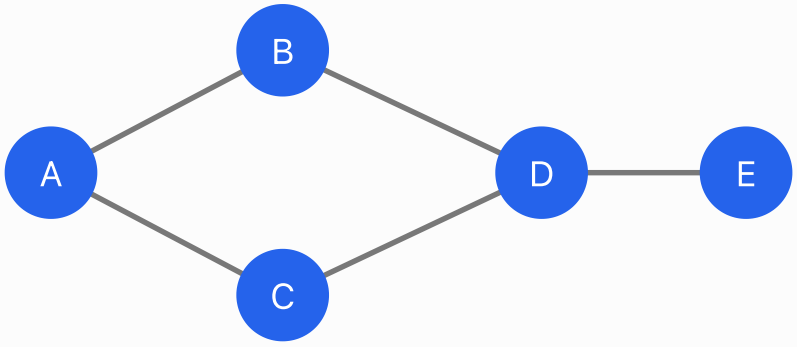
拓扑序列满足：

如果存在有向边：

$$u \to v$$

那么在拓扑序列中，u 必须位于 v 前面。

示例



有向边表示先后依赖关系。B 和 C 的相对顺序不受限制。

合法拓扑序列：

$$A, B, C, D, E$$

也可以：

$$A, C, B, D, E$$

因此拓扑序列不唯一。

2. 拓扑排序算法

使用入度进行排序。

步骤

1. 计算所有顶点的入度。
2. 找出入度为 0 的顶点。
3. 输出该顶点，并删除它发出的所有边。
4. 更新其他顶点的入度。

5. 重复直到所有顶点输出。

如果仍有顶点没有输出，但已经找不到入度为 0 的顶点，说明存在环。

代码模板

```c
int indegree[MAXV] = {0};
int queue[MAXV];
int front = 0, rear = 0;
int count = 0;

// 计算入度
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (G.Edge[i][j] != 0)
            indegree[j]++;
    }
}

// 入度为0的顶点入队
for (int i = 0; i < n; i++)
{
    if (indegree[i] == 0)
        queue[rear++] = i;
}
```

邻接矩阵实现：

$$O(n^2)$$

邻接表实现：

$$O(n+e)$$

3. 判断拓扑序列是否唯一

这是你之前做过的 408 算法设计题，非常值得记住。

核心结论：

在拓扑排序过程中，如果某一步同时出现两个或两个以上入度为 0 的顶点，那么拓扑序列不唯一。

原因：

这些顶点之间没有必须遵守的先后顺序，可以交换它们的输出顺序。

判断规则

当前入度为 0 的顶点数	说明
0 个，且尚未输出全部顶点	存在环
1 个	当前选择唯一
2 个及以上	拓扑序列不唯一

如果每一步都恰好只有一个入度为 0 的顶点，并最终输出全部顶点，则拓扑序列唯一。

408 算法设计题模板

```c

```

```
int uniquely(MGraph G)
{
    int n = G.numVertices;
    int indegree[MAXV] = {0};
    int queue[MAXV];
    int front = 0, rear = 0;

    // 计算入度
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (G.Edge[i][j] != 0)
                indegree[j]++;
        }
    }

    // 入度为0的顶点入队
    for (int i = 0; i < n; i++)
```

时间复杂度：

$$\boxed{O(n^2)}$$

七、关键路径（AOE 网）

关键路径是 408 计算题的重要内容。

1. AOE 网的基本概念

AOE 网：

用边表示活动，用顶点表示事件的带权有向无环图。

- 顶点：事件。
- 边：活动。
- 边权：活动持续时间。
- 源点：工程起点。
- 汇点：工程终点。

关键路径：

$$\boxed{\text{从源点到汇点的最长路径}}$$

关键路径长度就是完成整个工程所需的最短时间。

2. 四个核心时间参数

参数	含义
$ve(v)$	事件 v 的最早发生时间
$vl(v)$	事件 v 的最迟发生时间
$e(a)$	活动 a 的最早开始时间
$l(a)$	活动 a 的最迟开始时间

必须记住：

ve 取最大，vl 取最小。

3. 计算事件最早发生时间 ve

按照拓扑序列，从前往后计算。

初始化：

$$ve(v_0)=0$$

对于边：

$$v_i \xrightarrow{w} v_j$$

有：

$$\boxed{ve(v_j)=\max\{ve(v_j),ve(v_i)+w\}}$$

解释：

一个事件必须等所有前置活动完成后才能发生。

因此取最大值。

4. 计算事件最迟发生时间 vl

按照逆拓扑序列，从后往前计算。

假设工程总工期为 T：

$$vl(v_{end})=T$$

对于边：

$$v_i \xrightarrow{w} v_j$$

有：

$$\boxed{vl(v_i)=\min\{vl(v_i),vl(v_j)-w\}}$$

解释：

不能耽误后续活动，所以取最小值。

5. 计算活动时间

假设活动：

$$a:v_i \xrightarrow{w} v_j$$

最早开始时间

$$\boxed{e(a)=ve(v_i)}$$

最迟开始时间

$$\boxed{l(a)=vl(v_j)-w}$$

时间余量

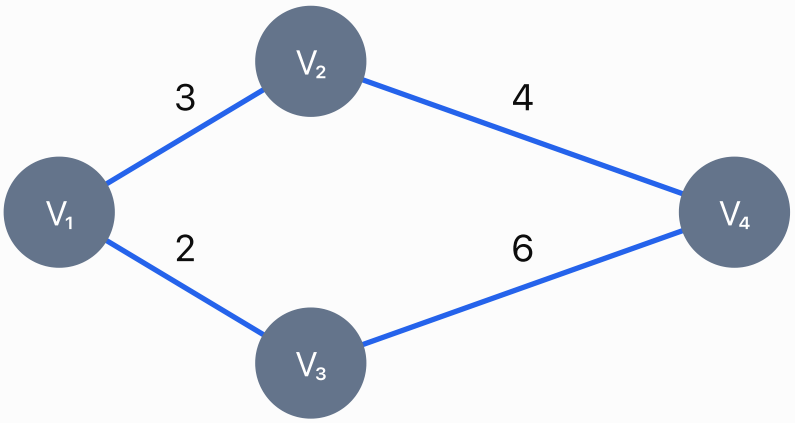
$$\boxed{d(a)=l(a)-e(a)}$$

判断：

$$\boxed{d(a)=0 \Rightarrow a \text{ 是关键活动}}$$

关键活动构成关键路径。

6. 完整例题



每条边旁的数字表示活动持续时间。

第一步：计算 ve

$$ve(v_1)=0$$

$$ve(v_2)=0+3=3$$

$$ve(v_3)=0+2=2$$

$$ve(v_4)=\max(3+4,2+6)=8$$

因此：

$$ve=[0,3,2,8]$$

工程最短工期：

$$\boxed{T=8}$$

第二步：计算 vl

从汇点倒推：

$$vl(v_4)=8$$

$$vl(v_2)=8-4=4$$

$$vl(v_3)=8-6=2$$

$$vl(v_1) = \min(4-3, 2-2) = 0$$

因此：

$$vl = [0, 4, 2, 8]$$

第三步：计算活动时间

活动	权值	e(a)	l(a)	d(a)
$V_1 \to V_2$	3	0	1	1
$V_1 \to V_3$	2	0	0	0
$V_2 \to V_4$	4	3	4	1
$V_3 \to V_4$	6	2	2	0

所以关键活动为：

$$V_1 \to V_3$$

$$V_3 \to V_4$$

关键路径：

$$\boxed{V_1 \to V_3 \to V_4}$$

长度：

$$2 + 6 = 8$$

关键路径的考试陷阱

1. 关键路径可能不止一条。
2. 所有关键活动不一定能够任意组合成关键路径。
3. 缩短一个关键活动的时间，不一定缩短总工期。
4. 如果存在多条关键路径，要缩短总工期，必须让所有当前关键路径都受到缩短，并注意可能出现新的关键路径。
5. 关键活动的时间余量为 0。

八、图算法的综合比较

这部分非常适合考前背诵。

算法	主要用途	核心思想	典型时间复杂度
DFS	图的遍历	深入到底再回退	$O(n+e)$
BFS	遍历、无权最短路径	逐层搜索	$O(n+e)$
Prim	最小生成树	逐步加入顶点	$O(n^2)$
Kruskal	最小生成树	按边权排序选边	$O(e \log e)$
Dijkstra	单源最短路径	每次确定最近顶点	$O(n^2)$
Floyd	所有顶点对最短路径	枚举中间顶点	$O(n^3)$
拓扑排序	DAG 依赖排序	不断删除入度为 0 的顶点	$O(n+e)$
关键路径	工程工期分析	正向取最大，逆向取最小	$O(n+e)$

注：DFS、BFS、拓扑排序和关键路径的复杂度按邻接表实现；Prim、Dijkstra 按常见邻接矩阵实现。

九、408 图的算法设计题模板

如果你的目标是提高算法设计题得分，我建议优先掌握以下五种套路。

题型 1：判断无向图是否连通

方法：一次 DFS，统计访问顶点数。

步骤：

1. 从任意顶点开始 DFS。
2. 标记所有能够到达的顶点。
3. 如果访问顶点数等于 n，则连通。
4. 否则不连通。

考试关键：

$$\boxed{\text{DFS访问顶点数} = n \Rightarrow \text{连通}}$$

题型 2：判断两个顶点之间是否存在路径

方法：DFS 或 BFS。

从起点搜索：

- 搜到目标顶点：存在路径。
- 搜索结束仍未找到：不存在路径。

复杂度：

邻接表：

$$O(n+e)$$

题型 3：判断图是否存在环

无向图

使用 DFS。

如果访问到一个已经访问过的邻接顶点，并且它不是当前顶点的父结点，则存在环。

有向图

使用拓扑排序。

$$\boxed{\text{拓扑排序输出顶点数} < n \Rightarrow \text{存在环}}$$

或者使用 DFS 三色标记法检测回边。

题型 4：判断拓扑序列是否唯一

方法：统计每一步入度为 0 的顶点数。

- 只有一个：继续。
- 多于一个：不唯一。
- 没有且未处理完：存在环。

这个题型建议直接记住代码模板。

题型 5：无权图最短路径

方法：BFS + 距离数组。

核心代码：

```c
dist[start] = 0;

while (!QueueEmpty(Q))
{
    int u = DeQueue(Q);

    for (每个u的邻接点v)
    {
        if (!visited[v])
        {
            visited[v] = true;

            dist[v] = dist[u] + 1;

            EnQueue(Q, v);
        }
    }
}
```

如果题目要求输出实际路径，可以增加前驱数组：

```c
pre[v] = u;
```

最后从目标顶点沿 `pre` 数组回溯即可。

十、考前速记：必须背下来的 20 个结论

408 图核心知识检查表 0/20 已掌握

1. 无向图所有顶点度数之和 = 2e
2. 有向图所有顶点入度之和 = 出度之和 = e
3. 无向完全图边数 = n(n−1)/2
4. 有向完全图边数 = n(n−1)
5. 无向连通图至少有 n−1 条边
6. 生成树恰好有 n−1 条边
7. 无向图奇数度顶点的个数一定为偶数
8. 邻接矩阵空间复杂度 O(n²)
9. 邻接表空间复杂度 O(n+e)
10. 有向图邻接矩阵：行出列入
11. 邻接矩阵的 k 次幂表示长度为 k 的步行数量
12. DFS 使用栈或递归，BFS 使用队列
13. BFS 可以求无权图最短路径
14. Prim 逐步扩充顶点集合
15. Kruskal 按边权从小到大选边，并避免成环
16. Dijkstra 不允许负权边
17. Floyd 的三层循环，k 在最外层
18. 拓扑排序要求有向无环图

19. 拓扑排序中同时有多个入度为 0 的顶点，则序列不唯一

20. 关键路径：ve 正向取最大，vl 逆向取最小；关键活动 d=0

十一、针对你目前的 408 复习，我建议这样安排

你现在已经开始做真题，而且希望通过整理常见题型来稳定拿分。所以图这一章，我建议按照以下优先级复习：

优先级	内容	目标
第一梯队	度与边数、邻接矩阵和邻接表	公式和复杂度必须记住
第一梯队	DFS、BFS	会判断序列，会写基本遍历
第一梯队	拓扑排序	会求序列、判环、判断唯一性
第一梯队	Dijkstra、Floyd	会手算过程，记住松弛公式

我尤其建议你把下面五个模板练熟：

1. **DFS/BFS 遍历模板**：解决连通、可达、遍历、无权最短路径。
2. **拓扑排序模板**：解决拓扑序列、判环、唯一性。
3. **Dijkstra 松弛公式**：解决单源最短路径。
4. **Kruskal + 并查集**：解决最小生成树。
5. **关键路径四个时间参数**：解决工程工期计算。

最后，我给你一个学习建议：

这是哪一类题 → 用哪个算法 → 套哪个步骤 → 注意什么陷阱。

尤其是拓扑排序那道 2024 年 408 算法设计题，你之前已经接触过。它就是非常典型的「掌握一个固定套路就可以处理一类题」的例子。

408 数据结构：树与二叉树复习笔记

这份笔记按照考研 408 的考查方式整理，重点是**基本性质、二叉树遍历、线索二叉树、树与森林、哈夫曼树、并查集**。

考虑到你现在主要通过真题和题型归纳复习，我会把知识点尽量整理成「**必背公式 → 解题步骤 → 典型例题 → 易错点**」的形式。

一、树的基本概念

1. 树的定义

树是由 n 个结点组成的有限集合，满足：

- 有且仅有一个根结点。
- 除根结点外，其余结点有且仅有一个双亲结点。
- 结点之间不存在环。
- 任意两个结点之间有且仅有一条简单路径。

一棵具有 n 个结点的树，边数一定为：

$$\boxed{e = n - 1}$$

考试记忆：树的边数永远比结点数少 1。

2. 树的基本术语

名称	含义
结点的度	该结点拥有的孩子结点数量
树的度	所有结点的度的最大值
叶子结点	度为 0 的结点
分支结点	度不为 0 的结点
双亲结点	某结点的直接前驱
兄弟结点	具有相同双亲的结点
祖先结点	从根到该结点路径上的其他结点
子孙结点	以某结点为祖先的结点
结点的层次	根为第 1 层，向下逐层加 1
树的高度	树的最大层数（根为第 1 层）
路径长度	路径上经过的边数

3. 树的重要公式

设：

- n：总结点数。
- n_i：度为 i 的结点数。
- m：树的度。

则：

$$\boxed{n = n_0 + n_1 + n_2 + \cdots + n_m}$$

所有结点的度数之和等于边数：

$$\boxed{n_1 + 2n_2 + 3n_3 + \cdots + mn_m = n - 1}$$

两式相减，可得到：

$$\boxed{n_0 = 1 + \sum_{i=2}^{m} (i-1)n_i}$$

这个公式可以解决一般树的叶子结点数量问题。

例题：

一棵树中，度为 2 的结点有 5 个，度为 3 的结点有 3 个，其他非叶结点的度均为 1，求叶子结点数。

$$n_0 = 1 + (2-1) \times 5 + (3-1) \times 3$$

$$\boxed{n_0 = 12}$$

注意：度为 1 的结点不影响叶子数量的计算。

二、二叉树的基本概念与性质

1. 二叉树的定义

二叉树是每个结点最多有两个孩子的树。

二叉树具有以下特点：

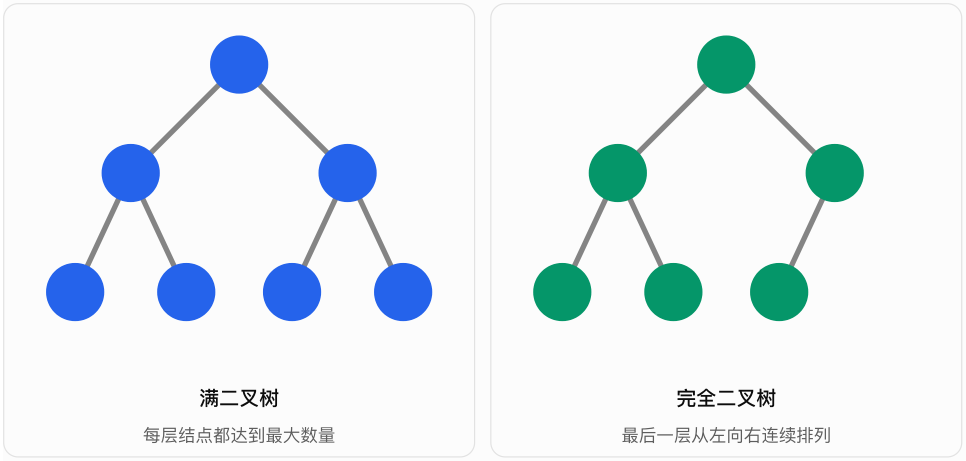
1. 每个结点最多有两个孩子。
2. 左子树和右子树有严格区别。
3. 可以只有左子树，也可以只有右子树。
4. 二叉树允许为空树。

特别注意：

二叉树不是简单地规定每个结点度不超过 2 的树。

普通树的孩子通常不区分左右，而二叉树区分左右次序。

2. 二叉树的分类



满二叉树

高度为 h 的满二叉树：

$$\boxed{n = 2^h - 1}$$

因此：

$$\boxed{h = \log_2(n+1)}$$

例如，高度为 4 的满二叉树：

$$n = 2^4 - 1 = 15$$

完全二叉树

完全二叉树的特点：

- 除最后一层外，其他层全部填满。
- 最后一层的结点全部集中在左侧。
- 可以直接使用数组按层序存储。

必背：满二叉树一定是完全二叉树，完全二叉树不一定是满二叉树。

三、二叉树的重要公式

这是 408 选择题的重点。

1. 第 i 层最多有多少结点?

规定根结点为第 1 层。

$$\boxed{2^{i-1}}$$

例如：

层数	最大结点数
第 1 层	1
第 2 层	2
第 3 层	4
第 4 层	8
第 5 层	16

2. 高度为 h 的二叉树最多有多少结点?

$$\boxed{n_{\max} = 2^h - 1}$$

最少有：

$$\boxed{n_{\min} = h}$$

注意，最少结点数为 h 是针对一般二叉树而言的。

3. 具有 n 个结点的完全二叉树高度

$$\boxed{h = \lfloor \log_2 n \rfloor + 1}$$

也可以写成：

$$\boxed{h = \lceil \log_2 (n+1) \rceil}$$

例如，完全二叉树共有 10 个结点：

$$h = \lfloor \log_2 10 \rfloor + 1 = 4$$

4. 二叉树叶子结点公式

设：

- n_0：度为 0 的结点数。
- n_1：度为 1 的结点数。
- n_2：度为 2 的结点数。

结点总数：

$$n = n_0 + n_1 + n_2$$

边数：

$$n - 1 = n_1 + 2n_2$$

相减：

$$\boxed{n_0 = n_2 + 1}$$

这是二叉树最重要的公式之一。

考试口诀：

二叉树的叶子结点数 = 双分支结点数 + 1。

例题

某二叉树中有 20 个度为 2 的结点，求叶子结点数。

$$n_0 = 20 + 1 = \boxed{21}$$

延伸结论

如果已知总结点数 n 和叶子结点数 n_0：

$$n_2 = n_0 - 1$$

$$\boxed{n_1 = n - 2n_0 + 1}$$

5. 完全二叉树的度数性质

完全二叉树中：

$$\boxed{n_1 \in \{0, 1\}}$$

也就是说：

度为 1 的结点最多只有一个，并且只能有左孩子。

对于具有 n 个结点的非空完全二叉树：

总结点数	度为 1 的结点数
n 为奇数	0
n 为偶数	1

叶子结点数：

$$\boxed{n_0 = \left\lceil \frac{n}{2} \right\rceil}$$

度为 2 的结点数：

$$\boxed{n_2 = \left\lfloor \frac{n-1}{2} \right\rfloor}$$

例如，完全二叉树有 10 个结点：

$$n_0 = 5, \qquad n_1 = 1, \qquad n_2 = 4$$

四、完全二叉树的顺序存储

完全二叉树适合使用数组存储。

1. 数组下标从 1 开始

对于编号为 i 的结点：

关系	计算公式
双亲结点	$\lfloor i/2 \rfloor$
左孩子	$2i$
右孩子	$2i+1$
最后一个分支结点	$\lfloor n/2 \rfloor$
第一个叶子结点	$\lfloor n/2 \rfloor + 1$

注意：

只有下标不超过 n 时，对应孩子才存在。

示例

若数组中存储 10 个结点：

结点 4：

$$parent(4) = 2$$

$$left(4) = 8$$

$$right(4) = 9$$

结点 5：

$$left(5) = 10$$

$$right(5) = 11$$

由于 $11 > 10$，因此结点 5 只有左孩子。

2. 数组下标从 0 开始

关系	计算公式
双亲结点	$\lfloor (i-1)/2 \rfloor$
左孩子	$2i+1$
右孩子	$2i+2$

考试一定先判断编号从 0 开始还是从 1 开始。

五、二叉树的链式存储

1. 二叉链表

每个结点包含：

1. 数据域。
2. 左孩子指针。
3. 右孩子指针。

```
C
```

```c
typedef struct BiTNode
{
    int data;

    struct BiTNode *lchild;
```

2. 空指针数量

一棵具有 n 个结点的二叉链表：

每个结点有两个指针域，因此总指针域数量：

$$2n$$

实际连接孩子的指针数量等于边数：

$$n-1$$

所以空指针数量：

$$2n-(n-1)$$

$$\boxed{n+1}$$

必背：具有 n 个结点的二叉链表，空指针域有 $n+1$ 个。

这是线索二叉树的基础。

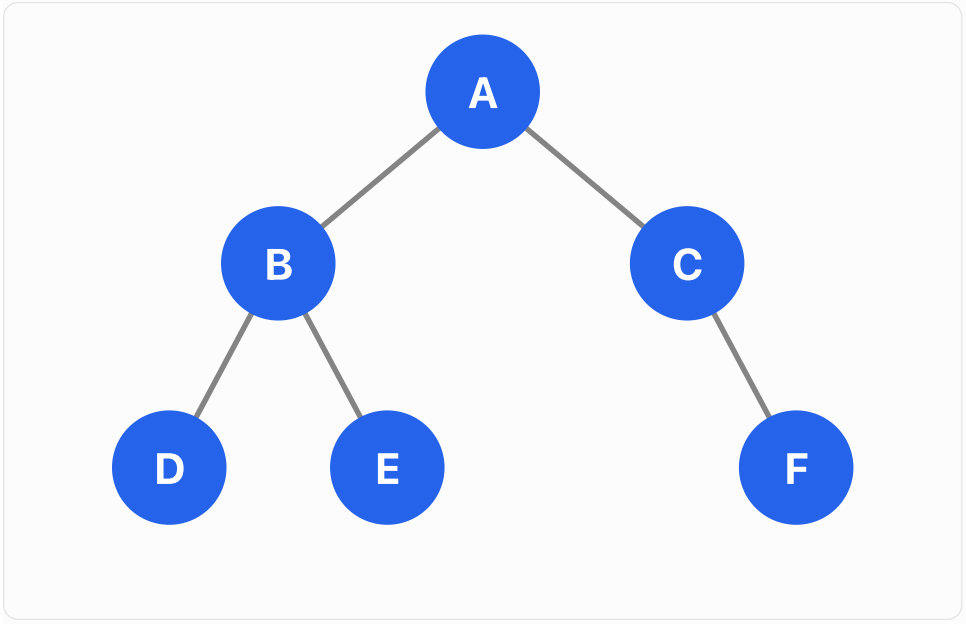
六、二叉树的遍历

二叉树遍历是 408 的核心考点。

主要有四种：

- 先序遍历。
- 中序遍历。
- 后序遍历。
- 层序遍历。

1. 遍历示例



先序遍历 NLR

访问顺序：

$$\boxed{\text{根} \rightarrow \text{左} \rightarrow \text{右}}$$

结果：

$$\boxed{A, B, D, E, C, F}$$

中序遍历 LNR

访问顺序：

$$\boxed{\text{左} \rightarrow \text{根} \rightarrow \text{右}}$$

结果：

$$\boxed{D, B, E, A, C, F}$$

后序遍历 LRN

访问顺序：

$$\boxed{\text{左} \rightarrow \text{右} \rightarrow \text{根}}$$

结果：

$$\boxed{D, E, B, F, C, A}$$

层序遍历

从上到下、从左到右。

结果：

$$\boxed{A, B, C, D, E, F}$$

2. 三种遍历的递归代码

先序遍历

```c
void PreOrder(BiTree T)
{
    if (T != NULL)
    {
        visit(T);

        PreOrder(T->lchild);

        PreOrder(T->rchild);
    }
}
```

中序遍历

```c
void InOrder(BiTree T)
{
    if (T != NULL)
    {
        InOrder(T->lchild);

        visit(T);

        InOrder(T->rchild);
    }
}
```

后序遍历

```c
void PostOrder(BiTree T)
{
    if (T != NULL)
    {
        PostOrder(T->lchild);

        PostOrder(T->rchild);

        visit(T);
    }
}
```

三种代码只需要记住：

`visit(T)` **放在什么位置。**

- 先序：放最前。
- 中序：放中间。
- 后序：放最后。

3. 遍历复杂度

三种遍历都会访问每个结点一次：

$$\boxed{T(n) = O(n)}$$

递归辅助空间：

$$\boxed{O(h)}$$

其中 h 为二叉树高度。

最坏情况下：

$$O(n)$$

平衡情况下：

$$O(\log n)$$

七、根据遍历序列还原二叉树

这是 408 经典题型。

1. 哪些序列能够唯一确定二叉树?

已知遍历序列	能否唯一确定
先序 + 中序	可以
中序 + 后序	可以
先序 + 后序	一般不可以
层序 + 中序	可以
只有先序	不可以
只有中序	不可以
只有后序	不可以

以上结论默认结点标识互不相同。

2. 先序 + 中序还原

核心步骤:

1. 先序第一个结点是根。
2. 在中序序列中找到根。
3. 根左侧为左子树。
4. 根右侧为右子树。
5. 递归处理。

例题

先序:

$$A, B, D, E, C, F$$

中序:

$$D, B, E, A, C, F$$

第一步:先序第一个为 A。

因此根为 A。

第二步:在中序中找 A。

$$DBE \mid A \mid CF$$

得到:

- 左子树:DBE。
- 右子树:CF。

第三步:继续递归。

左子树先序为:

$$BDE$$

左子树中序为:

$$DBE$$

因此 B 是根,D 和 E 分别为左右孩子。

右子树先序:

$$CF$$

右子树中序：

$$CF$$

因此 C 为根，F 为右孩子。

最终得到前面的二叉树。

3. 中序 + 后序还原

核心步骤：

1. 后序最后一个结点为根。
2. 在中序中找到根。
3. 划分左右子树。
4. 递归构造。

记忆：

> 先序看第一个，后序看最后一个，中序负责分左右。

4. 先序 + 后序为什么不能唯一确定?

例如：

先序：

$$AB$$

后序：

$$BA$$

可能对应：

- A 的左孩子为 B。
- A 的右孩子为 B。

两种二叉树的先序和后序完全相同。

特殊情况

如果已知二叉树是**真二叉树**，即每个结点的度只能是 0 或 2，那么先序和后序可以唯一确定二叉树。

八、线索二叉树

1. 为什么需要线索二叉树?

普通二叉链表有：

$$n+1$$

个空指针域。

这些空指针可以用于存储遍历序列中的：

- 前驱结点。
- 后继结点。

这样就形成了线索二叉树。

2. 线索二叉树的结构

通常增加两个标志位：

```c
typedef struct ThreadNode
{
    int data;

    struct ThreadNode *lchild;
    struct ThreadNode *rchild;

    int ltag;
    int rtag;

} ThreadNode;
```

标志含义：

标志	值	含义
ltag	0	lchild 指向左孩子
ltag	1	lchild 指向遍历前驱
rtag	0	rchild 指向右孩子
rtag	1	rchild 指向遍历后继

3. 线索二叉树的类型

- 先序线索二叉树。
- 中序线索二叉树。
- 后序线索二叉树。

主要区别：

前驱和后继是按照哪一种遍历顺序确定的。

4. 中序线索二叉树

假设中序遍历为：

$$D, B, E, A, C, F$$

那么：

结点	中序前驱	中序后继
D	无	B
B	D	E
E	B	A
A	E	C
C	A	F
F	C	无

注意：

只有对应的孩子指针为空时，才能把该指针改成线索。

中序后继查找规律

对于结点 p：

情况一：有右孩子。

中序后继是：

右子树中最左下的结点

情况二：没有右孩子。

如果 `rtag=1`，则 `rchild` 直接指向后继。

中序前驱查找规律

情况一：有左孩子。

中序前驱是：

左子树中最右下的结点

情况二：没有左孩子。

如果 `ltag=1`，则 `lchild` 直接指向前驱。

高频易错点

线索指针不是孩子指针。

因此，遍历线索二叉树时，必须判断标志位。

九、树、森林与二叉树的转换

408 主要考：

1. 树转换为二叉树。
2. 森林转换为二叉树。
3. 树与森林的遍历对应关系。

1. 树转换为二叉树

采用：

左孩子右兄弟表示法。

转换规则：

- 左指针指向第一个孩子。
- 右指针指向下一个兄弟。

例如，A 有三个孩子 B、C、D。

转换为二叉树后：

- A 的左孩子为 B。
- B 的右孩子为 C。
- C 的右孩子为 D。

记忆口诀：

> 左孩子，右兄弟。

2. 森林转换为二叉树

假设森林有三棵树：

$$T_1, T_2, T_3$$

转换规则：

1. 每棵树先转换为二叉树。
2. 第一棵树的根作为二叉树根。
3. 第二棵树的根作为第一棵树根的右孩子。

4. 第三棵树的根作为第二棵树根的右孩子。
本质上仍是：

左孩子右兄弟。

3. 遍历对应关系

树或森林的遍历	对应二叉树的遍历
树的先根遍历	二叉树先序遍历
树的后根遍历	二叉树中序遍历
森林的先序遍历	二叉树先序遍历
森林的中序遍历	二叉树中序遍历

这个表建议直接背下来。

十、哈夫曼树与哈夫曼编码

1. 哈夫曼树

哈夫曼树又称最优二叉树。

核心目标：

使带权路径长度 WPL 最小。

2. 带权路径长度 WPL

设叶子结点权值为 w_i，从根到该叶子的路径长度为 l_i。

$$\boxed{WPL=\sum_{i=1}^{n} w_i l_i}$$

注意：

路径长度是边数，不是结点数。

例题

叶子权值：

$$2,3,5,8$$

构造哈夫曼树。

第一步：选择最小的两个权值。

$$2+3=5$$

第二步：当前权值：

$$5,5,8$$

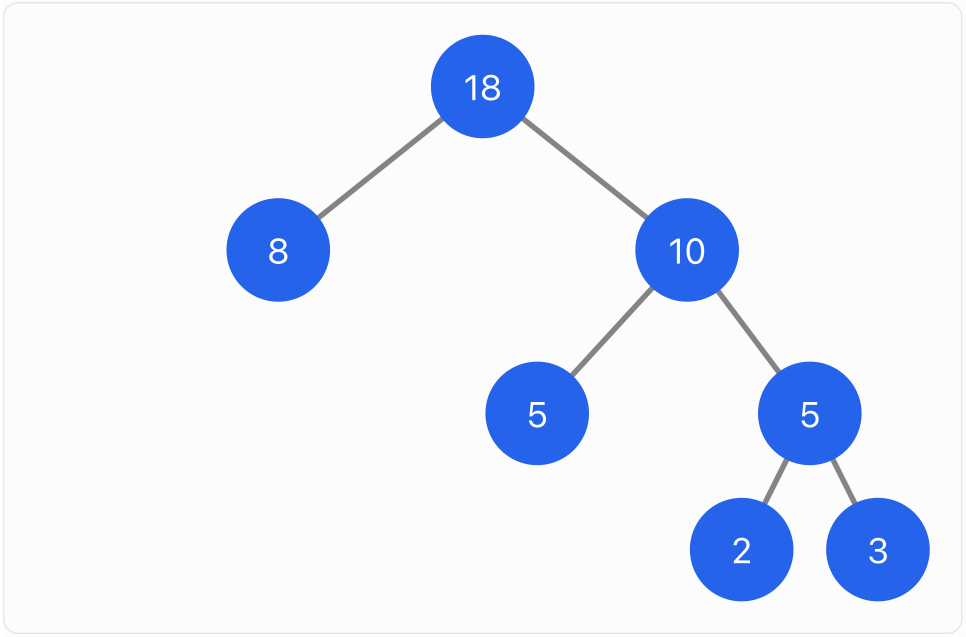
选择两个 5：

$$5+5=10$$

第三步：

$$8+10=18$$

最终根权值为 18。



计算 WPL：

- 8：深度为 1 条边。
- 5：深度为 2 条边。
- 2：深度为 3 条边。
- 3：深度为 3 条边。

$$WPL = 8 \times 1 + 5 \times 2 + 2 \times 3 + 3 \times 3$$

$$\boxed{WPL = 33}$$

也可以直接将构造过程中每次合并的权值相加：

$$\boxed{WPL = 5 + 10 + 18 = 33}$$

这个方法非常适合考试快速计算。

3. 哈夫曼树的性质

具有 n 个叶子结点的哈夫曼树：

- 不存在度为 1 的结点。
- 所有分支结点的度均为 2。
- 总结点数为 $2n - 1$。

$$\boxed{N = 2n - 1}$$

因为：

$$n_0 = n_2 + 1$$

所以：

$$n_2 = n - 1$$

$$N = n_0 + n_2 = n + (n - 1) = 2n - 1$$

4. 哈夫曼编码

哈夫曼编码是一种前缀编码。

编码规则：

- 左边通常标记为 0。
- 右边通常标记为 1。

从根到叶子的 0、1 序列就是对应字符的编码。

前缀编码特点

任何一个字符的编码都不是另一个字符编码的前缀。

因此可以无歧义地进行解码。

高频结论

1. 权值越大的字符，通常编码越短。
2. 权值越小的字符，通常编码越长。
3. 哈夫曼编码不一定唯一。
4. 哈夫曼树的 WPL 最小。
5. 哈夫曼编码属于变长编码。
6. 左右分支交换不会改变 WPL。

十一、并查集

并查集主要用于管理不相交集合。

408 中经常结合 Kruskal 算法、连通性判断考查。

1. 并查集的三个操作

操作	含义
Initial	初始化集合
Find	查找元素所属集合的根
Union	合并两个集合

2. 存储方式

通常使用数组：

```c
int parent[MAXN];
```

初始化：

```c
void Init(int n)
{
    for (int i = 0; i < n; i++)
    {
        parent[i] = -1;
    }
}
```

约定：

- `parent[i] < 0`：该结点是根，负值可表示集合大小。

- `parent[i] >= 0`：指向双亲结点。

3. Find 查找

普通版本：

```c
int Find(int x)
{
    while (parent[x] >= 0)
    {
        x = parent[x];
    }

    return x;
}
```

4. 路径压缩

路径压缩可以减少树的高度。

```c
int Find(int x)
{
    if (parent[x] < 0)
        return x;

    return parent[x] = Find(parent[x]);
}
```

核心：

将路径上的结点直接连接到根结点。

5. Union 合并

采用按规模合并：

```c
void Union(int x, int y)
{
    int rx = Find(x);
    int ry = Find(y);

    if (rx == ry)
        return;

    if (parent[rx] > parent[ry])
    {
        int temp = rx;
        rx = ry;
        ry = temp;
    }

    parent[rx] += parent[ry];
    parent[ry] = rx;
}
```

这里负数的绝对值表示集合大小，因此让规模较小的集合并入较大的集合。

并查集的复杂度

使用路径压缩和按规模合并：

单次操作的均摊复杂度：

$$\boxed{O(\alpha(n))}$$

其中 $\alpha(n)$ 是反阿克曼函数，增长极慢。

考试可以理解为：

均摊时间复杂度近似常数。

注意这并不意味着每一次 Find 的最坏时间都是 $O(1)$。

十二、二叉排序树 BST

虽然二叉排序树通常放在「查找」章节，但它和二叉树联系密切，建议一起掌握。

1. 定义

二叉排序树满足：

- 左子树所有结点的关键字小于根。
- 右子树所有结点的关键字大于根。
- 左右子树也分别是二叉排序树。

假设关键字互不相同。

最重要的性质

$$\boxed{\text{二叉排序树中序遍历得到递增序列}}$$

这是非常高频的选择题考点。

2. 查找

从根开始：

- 比根小：查左子树。
- 比根大：查右子树。
- 相等：查找成功。

查找复杂度取决于树高。

情况	时间复杂度
平衡较好	$O(\log n)$
退化为链表	$O(n)$

3. 插入

插入时按照查找规则：

1. 从根开始比较。
2. 小于根，进入左子树。
3. 大于根，进入右子树。
4. 找到空位置，插入新结点。

注意：

新插入的结点一定是叶子结点。

4. 删除

情况一：删除叶子结点

直接删除。

情况二：删除只有一个孩子的结点

使用该孩子代替被删除结点。

情况三：删除有两个孩子的结点

使用：

- 中序前驱，即左子树中的最大值。
- 或中序后继，即右子树中的最小值。

替换被删除结点，再删除原位置的前驱或后继结点。

十三、平衡二叉树 AVL

1. 定义

AVL 树是一种特殊的二叉排序树。

任意结点的左右子树高度差不超过 1。

平衡因子：

$$\boxed{BF = H_L - H_R}$$

合法平衡因子：

$$\boxed{BF \in \{-1, 0, 1\}}$$

2. 四种失衡类型

类型	失衡位置	调整方法
LL	左孩子的左子树	右旋
RR	右孩子的右子树	左旋
LR	左孩子的右子树	先左旋，再右旋
RL	右孩子的左子树	先右旋，再左旋

记忆：

- LL：右旋。
- RR：左旋。
- LR：先左后右。
- RL：先右后左。

3. LL 示例

插入：

$$30, 20, 10$$

形成：

```
纯文本

    30
   /
  20
 /
10
```

发生 LL 失衡。

对 30 右旋：

```
纯文本
```

```
   20
  /  \
 10   30
```

恢复平衡。

4. RR 示例

插入：

$$10, 20, 30$$

形成：

```
纯文本
```

```
10
  \
   20
     \
      30
```

对 10 左旋：

```
纯文本
```

```
   20
  /  \
 10   30
```

5. LR 示例

插入：

$$30, 10, 20$$

先对 10 左旋，再对 30 右旋。

6. RL 示例

插入：

$$10, 30, 20$$

先对 30 右旋，再对 10 左旋。

AVL 重要结论

AVL 树的高度：

$$\boxed{O(\log n)}$$

因此查找、插入、删除：

$$\boxed{O(\log n)}$$

十四、B 树与 B+ 树

这部分同样通常归在查找章节，是 408 的重要考点。

1. B 树

B 树是一种多路平衡查找树。

对于一棵 m 阶 B 树：

1. 每个结点最多有 m 棵子树。
2. 每个结点最多有 $m-1$ 个关键字。
3. 除根外的非叶结点，至少有 $\lceil m/2 \rceil$ 棵子树。
4. 非叶根结点至少有 2 棵子树。
5. 所有叶结点位于同一层。

特别注意：

m 阶表示最多 m 棵子树，不是最多 m 个关键字。

2. B+ 树

B+ 树主要特点：

- 非叶结点只保存索引关键字等导航信息。
- 数据记录通常存储在叶结点。
- 叶结点通常通过链表连接，方便范围查询。
- 查找路径通常到叶结点结束。

3. B 树与 B+ 树对比

比较	B 树	B+ 树
非叶结点	可存储数据记录	主要存储索引
查找成功位置	可在非叶结点	通常在叶结点
范围查询	相对不方便	方便
叶结点	不要求链式连接	通常链式连接
典型应用	多路查找	数据库索引

十五、408 常见题型与固定解法

这部分是我最建议你重点记忆的。

题型 1：求叶子结点数量

题目特征：

出现：

- 度为 0 的结点。
- 度为 1 的结点。
- 度为 2 的结点。

直接套公式：

$$\boxed{n_0 = n_2 + 1}$$

如果是一般树：

$$\boxed{n_0 = 1 + \sum_{i=2}^{m}(i-1)n_i}$$

题型 2：求完全二叉树的高度

已知总结点数 n。

$$\boxed{h = \lfloor \log_2 n \rfloor + 1}$$

例如：

$$n = 100$$

$$h = \lfloor 6.64 \rfloor + 1 = \boxed{7}$$

题型 3：求完全二叉树孩子编号

先判断数组从几开始编号。

如果从 1 开始：

$$\boxed{left = 2i}$$

$$\boxed{right = 2i + 1}$$

$$\boxed{parent = \lfloor i/2 \rfloor}$$

题型 4：求二叉链表空指针数量

$$\boxed{n + 1}$$

无需画图。

题型 5：遍历序列

看到先序、中序、后序：

- 先序：根左右。
- 中序：左根右。
- 后序：左右根。

建议在草稿纸画出树，再按照顺序遍历。

题型 6：重建二叉树

已知先序和中序：

1. 先序找根。
2. 中序分左右。
3. 递归构造。

已知中序和后序：

1. 后序找根。
2. 中序分左右。
3. 递归构造。

题型 7：求哈夫曼树 WPL

步骤：

1. 每次选择两个最小权值。
2. 将两者合并。
3. 把合并结果重新加入集合。
4. 重复直到只剩一个结点。
5. 所有合并值相加。

$$\boxed{WPL = \text{所有合并权值之和}}$$

题型 8：判断二叉排序树

最简单的方法：中序遍历。

如果得到严格递增序列，则满足二叉排序树性质（关键字互不相同时）。

题型 9：AVL 旋转

先判断失衡类型：

- LL → 右旋。
- RR → 左旋。
- LR → 先左旋后右旋。
- RL → 先右旋后左旋。

题型 10：并查集

看到：

- 集合合并。
- 连通性判断。
- 判断是否属于同一集合。
- Kruskal 算法判环。

优先想到并查集。

十六、408 树与二叉树必背公式汇总

考点	公式
树的边数	$e = n - 1$
树的度数之和	$\sum d_i = n - 1$
二叉树叶子数量	$n_0 = n_2 + 1$
第 i 层最多结点数	2^{i-1}
高度 h 的二叉树最多结点	$2^h - 1$
完全二叉树高度	$\lfloor \log_2 n \rfloor + 1$
完全二叉树叶子数量	$\lceil n/2 \rceil$
完全二叉树度为 2 的结点数	$\lfloor (n-1)/2 \rfloor$
二叉链表空指针数	$n + 1$
满二叉树结点数	$2^h - 1$
哈夫曼树总结点数	$2n - 1$，其中 n 为叶子数
哈夫曼树 WPL	$\sum w_i l_i$
AVL 平衡因子	$H_L - H_R$
AVL 平衡条件	$\lvert BF \rvert \le 1$
m 阶 B 树最多关键字	$m - 1$

十七、针对你当前复习的优先级

考虑到你现在更希望把常见题型吃透、争取稳定拿分，我建议这样分配精力：

优先级	知识点	掌握目标
必会	二叉树性质与公式	看到题能直接套公式
必会	二叉树四种遍历	能手算遍历序列
必会	遍历序列还原二叉树	能独立画出二叉树
必会	哈夫曼树	会构造、求 WPL、求编码
必会	二叉排序树	会插入、删除、判断

最后给你一个特别实用的建议：

这一章不要只背定义，一定要练「手算过程」。

尤其是下面六种题，我认为非常值得集中练习：

1. 给出结点个数，计算完全二叉树高度、叶子数和孩子编号。
2. 给出二叉树，计算先序、中序、后序遍历。
3. 给出两种遍历序列，还原二叉树。
4. 给出权值，构造哈夫曼树并计算 WPL。
5. 给出关键字序列，构造 BST、AVL 树。
6. 给出 B 树的关键字插入序列，判断分裂结果。

对于这六种题型，你完全可以采用之前整理数学二的思路：**每一种题型总结一套固定步骤，再针对性练习 3～5 道题，直到不看答案也能完整做出来。**